

开发规范与 AI 协作方式 v1

2026-09-26 | 适用：2 名开发者（含各自使用的 AI 助手） | 配套：CONTRIBUTING.md、docs/collaboration/2026-09-25-协作者验证包.md、docs/planning/2026-09-26-v2-项目规划-v1.md

这份文档解决一件事：两个人（以及他们各自的 AI）并行开发时，不打架、不互相污染、不把“没验证”当“已完成”。

1. 四条基本约定（先记住这四条，其余都是细节）

#	约定	为什么
1	一个 Issue = 一个分支 = 一个 PR，不串任务	串起来就无法独立验收，出问题也不知道是谁的
2	main 只进 PR，谁都不许直接推	保护线：main 永远可运行、门禁永远绿
3	没有证据 = 没完成	"我实现了"不算；要命令 + 原始输出 + 反例
4	冻结资产只调用不修改	内核、lineage-client、dip-contracts、dip-pg 是地基，动它们要单独开 Issue

2. 仓库与分支

main ← 只接受 PR；始终可运行、门禁绿

└─ feat/m1-03-lineage-skill 功能（对应 Issue #3）

└─ fix/m2-04-cjk-policy 修缺陷

└─ docs/collab-ai-rules 文档

└─ chore/bump-ruff 杂务

- 分支名 = <类型>/<issue 编号或里程碑>-<简述>；分支活不过一周，大了就拆。
- 开工前必须 git pull --rebase；推送前 git fetch && git rebase origin/main（保持线性，方便回溯）。
- 合并方式：Squash merge（一个 Issue 在 main 上只留一个提交，便于回滚）。
- 禁止 git push --force（除非是你自己未合并的分支，且写明原因）。

3. 提交信息

<类型>(<范围>)：<一句话说清做了什么>

- 关键点 1（为什么这么改，而不是怎么改）

- 关键点 2

Closes #3

类型：feat / fix / docs / refactor / test / chore / perf。

要求：一次提交只做一件事；中文正文；必须关联 Issue（Closes #N 或 Refs #N）。

4. 代码规范（Python 为主，前端待开工时补）

4.1 通用

规则	说明
格式化与静态检查	<code>ruff</code> （配置在 <code>pyproject.toml</code> ，行宽 118，目标 py310）； <code>ruff check</code> 必须干净
命名	标识符用英文，docstring / 注释用中文；不要拼音（ <code>chanliang</code> 这种是内核历史包袱，新代码别学）
函数长度	超过 60 行就拆；一个函数只做一件事
类型标注	所有公共函数必须有类型标注； <code>from __future__ import annotations</code> 打头
日志	库代码用 <code>logging</code> ； <code>print</code> 只允许出现在 CLI / 脚本 / 启动提示里
异常	不许裸 <code>except</code> ；要么处理要么抛出，捕获了就要记录并说明原因
注释	只写“为什么”，不写“是什么”（代码本身说明是什么）；反直觉的处理必须写原因

4.2 分层铁律（有测试强制，违反了会红）

```
apps/portal-api → packages/dip-agent → packages/dip-core / dip-contracts
apps/portal-api → integrations/lineage-client
dip-contracts / dip-core / lineage-client → **不得**依赖上层
```

- 契约层（`dip-contracts`）只许用 `pydantic` + 标准库（`tests/test_layering.py` 会拦）。
- 只有 API 层知道数据库；`dip-agent` 不许出现 SQL / 连接串。

4.3 数据与世界观的硬规矩

规矩	后果
事实不许编：结论必须能指到来源（文件/端点/行号）	拿不到就说“没查到”，不许编一个像样的回答
无凭证不出结论	<code>evidence</code> 为空时必须不出数字/公式（有测试断言）
中文标识符要 Unicode 感知	表名/字段名含中文，用 <code>[A-Za-z_]\w*</code> 会漏检——测试还会假绿
测试不许“为了让通过率好看”改断言	改断言 = 改需求，必须走 Issue 讨论
不许提交密钥	本地 <code>.env</code> （已 gitignore），仓库只留 <code>.env.example</code>
不许改 CI / 门禁规则来绕过红灯	门禁是共同底线，改它要单独 Issue + 两人同意

4.4 SQL 与数据开发侧（Windows 端做数据开发时同样适用）

- 脚本命名：`t_<表名>.sql`；每个脚本头部注释里写清这段 SQL 的业务含义——注释就是口径的原材料，会被知识提炼流水线读走。
- 建表/改表脚本必须能被解析器解析（别写方言扩展或动态拼接），否则血缘解析会漏。
- demo 数据与生产数据物理隔离；任何情况下不许把公司生产数据带出来跑（这条是红线，没有例外）。
- 口径变更走候选池 → 责任人审核 → 版本化入库，不许直接改知识库。

5. 测试规范

要求	说明
----	----

新功能必须带测试	没有测试的功能不算完成
必须有反例	至少一条"该被拒绝/该失败"的用例；反例比正例值钱
无依赖优先	用录制 fixture (integrations/lineage-client/tests/fixtures/) 而不是真起内核
需要外部服务就跳过	<code>pytest.mark.skipif</code> + 明确的 <code>skip</code> 原因 (不许静默失败)
测试必须幂等	用唯一 id (如 <code>uuid4</code>) 而不是固定会话名——固定 id 会在同库重复跑时累积数据、导致假红 (我们踩过)
慢测试分类	需要真内核/真库的打 <code>smoke</code> 标记, <code>make smoke</code> 单独跑

6. 门禁（提交前的唯一标准）

```
bash ops/gate.sh          # lint + 测试 → 必须看到 GATE: ALL_PASS
bash ops/gate.sh --with-smoke  # 带上真内核的冒烟
```

- **红了不许提 PR**（这是我们两次踩过的坑：红灯推上去，reviewer 白跑）。
- 本地门禁与 CI 跑的是同一条命令；**CI 红而本地绿 = 环境差异**，必须在 PR 里说明并修。
- Windows 用户：脚本自动探测 `.venv/Scripts` 与 `.venv/bin`；**HTTP 验证必须显式 UTF-8**（否则中文变乱码，会被误判成产品缺陷）。

7. AI 协作方式（本次新增的重点）

7.1 给 AI 的任务上下文（避免"上下文的公地悲剧"）

做法	说明
一个 Issue 一个会话	别把整个项目历史灌进一个会话；Issue 描述 + 相关 ADR 就是上下文
AI 先读三份	<code>CONTRIBUTING.md</code> → 相关 Issue → 本文件 (+ 涉及的 ADR)
明确"边界"再开工	把 Issue 里的 边界（不要做什么） 原样贴给 AI，它最爱扩范围
让它先复述	"用三句话复述你要做什么、边界在哪、验收是什么"——复述错了就别让它动手

7.2 AI 的权限边界（写进提示词）

允许：读改本仓库代码、写测试、跑命令、提 PR（草稿）。

必须人来做：合并 PR、改 CI / 门禁规则、改冻结资产、处理凭证、`force push`、推翻既有决策（ADR）。

绝对禁止：把没跑通的验证说成已验证；伪造输出/截图；为了让门禁变绿而改测试或门禁；在仓库里写密钥；把生产数据带出来。

7.3 AI 交付的"证据三件套"（缺一不可）

- **命令 + 原始输出**（关键行不截断，贴的是真跑的结果）
- **测试文件路径 + 通过输出**（含至少一条反例）
- **边界声明：**明确说"我没有做 X（按边界要求）"，让人知道哪里没被覆盖

一句话：**AI 说"完成了"不算数，能被我复跑出同样结果才算数。**

7.4 人审责任（不可转移）

- AI 生成的代码，**提交人必须能逐行解释**——解释不了的代码不许合并。
- PR 里新增一栏 "**AI 参与度**"：哪些部分由 AI 生成/改写、哪些是人工重写、你亲手验证了哪几条。
- 这是诚信要求，不是形式：审查时我会抽查提问（"这行为什么这么写"）。

7.5 两个人 + 两个 AI 的协作纪律

场景	规则
同一个文件	同一时间只允许一个人（或一个 AI）改 ；开工前在 Issue 里留言"我在改 X 文件"
接口未定	契约先行 ：先定接口（哪怕只写签名 + 文档），再并行实现；不许各写一套
发现别人的坑	不要顺手改 ：开 Issue 或在 PR 里留言；顺手改会造成 review 冲突
大改动	先开 Issue 写方案（半页），对方点头再动手
卡住了	24 小时内上报（格式见 §9），不要闷头硬撑到周末
冲突处理	<code>git rebase</code> 解决，不许 <code>-X ours/theirs</code> 一把梭；解决完必须重跑门禁

7.6 每日同步（异步，各 5 分钟）

每天在**自己的 Issue** 下留一条进度评论（不要发在群里，信息会淹）：

进展：做了什么（带证据路径）
计划：下一步做什么
卡点：卡在哪 / 需要谁配合（没有就写“无”）

8. 验收与合并

验收人：项目负责人（可让我代跑）。流程：

- 提交 PR（描述里写清"怎么验证"，并填 AI 参与度）
- 自动门禁绿 → 我方逐条跑 Issue 里的验收标准 → **逐条给"通过/不通过 + 证据"**
- 全通过才合并（Squash）；不通过**打回并写明原因**（哪一条没过、期望什么）

打回的标准写法（对事不对人）：

验收不通过：<编号> <验收条目>
实际：<命令 + 输出>
期望：<Issue 里写的标准>
建议：<具体怎么改>

9. 卡点上报格式（谁都能看懂）

【卡点】<Issue 编号> <一句话>
现象：<可复现命令 + 原始报错>
已排查：<试过什么、排除了什么>

怀疑：<你的判断，可以是错的>
需要：<谁的什么决定 / 什么权限 / 什么信息>

10. 提交前自查清单（贴在显示器上）

- `[]` `bash ops/gate.sh` → `GATE: ALL_PASS`
- `[]` 关联了 Issue (`Closes #N`) , 分支名合规
- `[]` 有新测试, **且至少一条反例**
- `[]` 没碰冻结资产; 没改 CI / 门禁规则
- `[]` 没有密钥进入提交 (`git diff --cached | grep -i "sk-\\|token\\|password"` 为空)
- `[]` PR 描述含"怎么验证" + AI 参与度
- `[]` 边界确认: 只改了 Issue 范围内的东西
- `[]` 中文相关改动**有中文用例** (标识符 / 动作名 / 提示语)

11. 与既有文档的关系

文档	管什么
本文件	两个人 (和他们的 AI) 怎么一起写代码
<code>CONTRIBUTING.md</code>	入口与工作流 (最短路径)
《协作者验证包 v1》	具体任务怎么上手、每条任务的验收标准
《v2 项目规划 v1》	做什么、分几个里程碑、每个工作项的边界
<code>docs/adr/</code>	已定架构决策 (改它要走 Issue)
<code>_AI_PROMPT_CARDS.md</code> (同目录)	可直接复制给 AI 的提示词